

DemoTrack

Build Plan — Phases & Detailed Tasks

Companion to the Project Specification v3.1 · Tunis, Tunisia · 2026

This is the engineering checklist for building DemoTrack. It breaks every phase into concrete, checkable tasks, with what each phase depends on and what “done” means. Phases are built in order so the app is **usable early** — by the end of Phase 4 you can send and record a real demo — and grow in value from there. The Contact-Database research (Phase 0) runs in parallel throughout.

How to use this. Work top-to-bottom within each phase; the ☐ items are designed to be ticked off. A phase is shippable on its own, so deploy at the end of each. “Size” in the overview is a rough effort signal (S / M / L), not a deadline.

PHASE OVERVIEW

Phase	Size	Outcome	Milestone
0 · Contact DB Research	Parallel	A real, qualified label database	ongoing
1 · Foundation	L	Auth + schema + deploy	—
2 · Track Vault	S	Store your tracks	—
3 · Contacts CRM	M	Labels live in-app	—
4 · Send Demo	M	Send + record a demo	★ first usable
5 · Email Presets + AI Hook	M	1-minute tailored email	—
6 · Follow-up + Digest	L	Told who to chase, reminded	—
7 · Feedback Log	S	Every reply + silence captured	—
8 · A&R Intel + Prioritization	M	Ranked by history, intel attached	—
9 · Dashboard + Funnel	M	“What now?” + conversion	—
10 · Label Discovery	M	Find next target, add to CRM	—
11 · Link Tracking (optional)	M	See Form/DM opens	—
12 · Work Sessions + Goals	S	Accountability, derived	—
13 · Artist Press Kit	M	Auto-attaching EPK	—

Phase 0 · Contact Database Research

Parallel track — runs throughout the whole build. The app is the engine; this database is the fuel.

Goal	Build and maintain a curated, verified library of career-changing labels + promo contacts.
Depends on	Nothing — start immediately, alongside Phase 1.
Done when	The labels table is populated and feeds Discovery & the CRM.

SET UP

- Create a research tracking sheet with columns matching the labels schema (name, tier, access_path, method, link, genre_tags, bpm range, 2 sources, last_verified, submission_requirements, parent_label_id, notes).
- Write the qualification rubric as a checklist: the **≥3-of-5 bar** (Beatport chart presence · booking impact · streaming credibility · scene respect · accessible to unknowns).

THE RESEARCH LOOP (PER CONTACT)

- **Source** — mine Beatport charts, label rosters, artist credits, scene press, and submission directories; include non-label channels (radio, blogs, playlist/repost curators, premiere channels).
- **Verify** — find the real submission route (email / portal / DM) and confirm it against **2 independent sources**; discard dead inboxes and outdated forms.
- **Qualify** — apply the $\geq 3/5$ bar; if it fails, it does not go in.
- **Tier + access_path + priority** — assign Elite / A / B and a realistic access path (cold-demo-friendly / open-window-only / needs-warm-intro / relationship-only).
- **Record** — capture submission requirements (format, unreleased-only, single-vs-album), genre/BPM fit, the contact link, the 2 sources, and an A&R-intel note.
- **Capture sub-label links** — note efficiency multipliers (1 Defected $\approx$ 11 sub-labels; 1 Repopulate Mars $\approx$ 3) via parent_label_id.
- **Maintain** — stamp last_verified; set a re-check cadence so Discovery can flag stale entries.

MIGRATION

- Export the sheet to CSV in the exact labels column order.
- Import ≈ 170 verified labels + ≈ 98 non-label promo contacts into the labels table (Supabase CSV import or a seed script).
- Re-confirm the 17 curated seed labels are present and correctly tiered.

Definition of done (ongoing). Never fully finished, but the in-app database is ready to drive Discovery and the CRM once migrated, and a maintenance cadence is in place.

Phase 1 · Foundation

Goal	Stand up the project: data, auth, security, app shell, and deployment.
Depends on	Nothing (and Phase 0 in parallel).
Done when	You can log in via magic link, your data is private by RLS, and the app is deployed.

PROJECT SETUP

- ☐ Create the Supabase project; record the project URL and anon/service keys.
- ☐ Scaffold the frontend: **Vite + React 19 + TypeScript**.
- ☐ Install and configure **Tailwind CSS 4**; set a base theme (brand purple, mobile-first defaults).
- ☐ Create .env for SUPABASE_URL, SUPABASE_ANON_KEY; keep ANTHROPIC_API_KEY **server-side only** (never in the client bundle).
- ☐ Add the Supabase JS client and a session/auth context.

DATABASE SCHEMA (12 TABLES)

- ☐ Write SQL migrations for all 12 tables: contacts, tracks, submissions, feedback, templates, ar_intel, work_sessions, goals, link_events, press_kit, labels, notifications.
- ☐ Define enums / controlled vocabularies: **genre_tags**, track **status**, submission **status** & **method**, **access_path**, **relationship_stage**, **mood**, **goal type/timeframe**, feedback **response_type**.
- ☐ Add foreign keys (e.g. contacts.label_id → labels.id, submissions.track_id/contact_id, tracks.exclusive_hold_contact_id).
- ☐ Add a user_id (owner) column to every user-owned table, defaulting to auth.uid().

SECURITY (RLS)

- ☐ Enable Row-Level Security on every table.
- ☐ Write owner-only policies: user_id = auth.uid() for select/insert/update/delete on all personal tables.
- ☐ Make labels read-only to authenticated users (it's a shared library, no per-row owner).
- ☐ Create the **public press-kit RPC**: a SECURITY DEFINER function that takes a slug and returns only whitelisted public fields — the public page never reads the table directly.

APP SHELL & AUTH

- ☐ Configure **magic-link (email OTP)** auth in Supabase; build the login screen.
- ☐ Set up routing (React Router) with a protected-route guard and a main layout/nav.
- ☐ Add the **PWA**: web manifest, service worker, installable, mobile-first responsive base.

DEPLOY

- ☐ Deploy to **Vercel**; set environment variables in the Vercel project.
- ☐ Connect the demotrack.app domain (and reserve the press. / track. subdomains).
- ☐ Verify: sign in via magic link end-to-end; confirm RLS blocks cross-account reads.

Definition of done. Log in via magic link, see an empty but secured account, on a deployed URL.

Phase 2 · Track Vault

Goal	Your catalogue: store tracks, statuses, listen links, and holds.
Depends on	Phase 1.
Done when	You can store your tracks and see their status.

TASKS

- ☐ Build the track **list view**: status badges, sort, and filter by status/genre.
- ☐ Build the **add/edit form**: title, genre tags (multi-select from the vocab), BPM, key, status, notes.
- ☐ Add `listen_link + link_type` (soundcloud-private / dropbox / other); **audio is never uploaded** — only the link is stored.
- ☐ Implement the **readiness checklist** (mastered? · correct format? · metadata/tags? · artwork?) shown before a track becomes “demo-ready.”
- ☐ Add **exclusive-hold** fields (`exclusive_hold_contact_id`, `hold_until`) with a clear on/off display.
- ☐ Wire the status workflow idea → demo-ready → submitted → signed with valid transitions.
- ☐ Add empty states and form validation.

Definition of done. Add, edit, and list tracks with status; set a hold on a track.

Phase 3 · Contacts CRM

Goal	Your people: every label/contact with submission details and history.
Depends on	Phase 1 (and Phase 0 data).
Done when	Your label list lives in the app with submission details.

TASKS

- ☐ Build the contacts **list** with the 7 categories (Labels, DJs, A&Rs, Curators, Blogs, Promoters, Radio) and category/access-path filters.
- ☐ Build the contact **form**: name, category, submission method, email/portal/DM link, `relationship_stage`, `access_path`, notes.
- ☐ **Seed the 17 curated labels** into `labels`, and provide a way to add them to your contacts.
- ☐ Build the contact **detail view** with a full history panel (submissions + feedback) — empty for now, wired to populate later.
- ☐ Display the derived **“times contacted”** count on each contact (count of submissions).

Definition of done. Your label list lives in the app with submission methods and a history view.

Phase 4 · Send Demo

★ *The first usable milestone — after this, the app pays you back.*

Goal	The submission flow: pick, prepare, send, record — under 3 minutes.
Depends on	Phases 2 & 3.
Done when	You can send a demo and it's recorded.

TASKS

- ☐ Build the flow: **select track** → **select contact** → **choose preset** (preset is a stub here; the engine arrives in Phase 5).
- ☐ Implement **method routing** with color-coded badges: Email  / Form  / DM .
- ☐ **Email** — build a `mailto:` / Gmail-compose deep link with prefilled subject + body; opens your own inbox.
- ☐ **Form** — open the portal/platform in a new tab.
- ☐ **DM** — open the profile in a new tab.
- ☐ Render the **pre-send checklist** from the contact/label `submission_requirements`.
- ☐ Add the **exclusive-hold warning** — block/warn if the chosen track is held by another label.
- ☐ On send, **record a `submissions` row** (status = sent, method, dates).
- ☐ Add the **confirm step** for Form/DM (“Did you send it?”) so it's logged accurately.
- ☐ Add the **auto-attach press-kit link** toggle (stubbed until Phase 13).

Definition of done. Send a demo through any of the 3 methods and have it recorded in history.

Phase 5 · Email Presets + AI Hook

Goal	Templates that auto-fill, with AI suggesting only the one personal sentence.
Depends on	Phase 4.
Done when	A tailored, personal demo email in ~1 minute, sent from your own inbox.

PRESETS & MERGE FIELDS

- ☐ Build **preset CRUD** (a template editor) storing skeletons with `{merge_fields}` and one `{hook}` slot.
- ☐ Build the **merge-field engine**: parse `{...}` and auto-fill from the track, contact, and your profile.
- ☐ Seed starter presets: **cold email**, **DM** (shorter), **warm follow-up**, **form-portal note**.

AI HOOK

- Create a **serverless/Edge function** that calls the Anthropic API (key stays server-side).
- Write the **hook prompt** with guardrails: ≤ 30 words, **never fabricate** a release/stat/angle, use ar_intel only if present, otherwise return an honest generic line.
- Build the hook UI: generate → show 1-2 options → edit → **required** validation → **duplicate-send warning**.
- Add output handling + a plain-text fallback if the model's formatting varies.
- **Wire it into Send Demo**: the assembled subject + body now flow into the mailto/compose link.

Definition of done. Pick track → pick label → confirm one AI-suggested sentence → send, in about a minute.

Phase 6 · Follow-up + Notifications Digest

Goal	Tell you who to chase, let you close the loop in one tap, and reach you when the app is shut.
Depends on	Phase 4 (submissions exist).
Done when	The app tells you who to follow up and reminds you outside the app.

STATUS & QUEUE

- Implement the `submissions.status` state machine: sent → opened → replied → considering → signed → passed.
- Build the **follow-up queue** showing only sent/opened items with days-since.
- Add **one-tap actions** per row: **“Got a reply”** (opens a 5-second feedback log) and **“Still silent”** (keeps nudging).

AUTOMATION (PG_CRON)

- Schedule a job that queues a **7-day silence** nudge.
- Schedule a job that flags **14-day overdue** and **auto-logs a `no-response` feedback record**.
- Build follow-up preset templates (reuse the Phase-5 engine).

DIGEST

- Create the notifications table + user prefs (channel, cadence).
- Build a **Scheduled Edge Function** that composes the daily/weekly digest (“3 follow-ups due, 1 demo seen”).
- Send the digest **to your own inbox** (Resend or Supabase SMTP) — note this is self-notification, so deliverability is a non-issue.

Definition of done. The overdue queue is accurate, one tap advances a send's status, and a scheduled digest lands in your inbox.

Phase 7 · Feedback Log

Goal	Capture every reply — and every silence — as data.
Depends on	Phase 6.
Done when	Every reply and non-reply is captured and searchable.

TASKS

- Build **feedback CRUD** tied to a submission (and thus a track + contact).
- Wire the **one-tap reply logging** from the follow-up queue: response-type dropdown + optional pasted text.
- Surface the **auto `no-response` records** (created by Phase-6 cron) in the log.

- Define the **response-type taxonomy**: signed · pass · not-for-us · constructive · no-response · other.
- Add list + search and back-links to the track and contact.

Definition of done. Every reply and silence is captured, searchable, and linked back to its track and contact.

Phase 8 · A&R Intel + Prioritization

Goal	Attach research to each contact and surface untried targets first — no scoring engine.
Depends on	Phases 3 & 5.
Done when	Contacts show history at a glance, with intel attached and untried targets first.

TASKS

- ☐ Build **A&R-intel CRUD** (one record per contact): key people, what they sign now, dos & don'ts, recent releases, your notes.
- ☐ **Feed `ar\_intel` into the Phase-5 hook prompt** so the AI's sentence is informed.
- ☐ Display the **“times contacted”** count prominently on labels and contacts.
- ☐ Implement the **prioritization sort**: cold-demo-friendly + untried (0 contacts) first, then by tier.
- ☐ Confirm there is **no weighted score** — prioritization is access_path + times-contacted only.

Definition of done. Each contact shows its history and intel, and untried, reachable labels rise to the top.

Phase 9 · Dashboard + Funnel

Goal	The home screen that answers “what should I do right now?” plus your real conversion.
Depends on	Phases 4, 6, 7.
Done when	The home screen aggregates every signal and shows the funnel.

TASKS

- ☐ Build the **dashboard** aggregating: sends due, follow-ups due today, recent opens/replies, goal progress, recent feedback, work-session streak.
- ☐ Build the **funnel** view (sent → opened → replied → considering → signed) backed by **Postgres views**.
- ☐ Add **response-rate by genre/tier**.
- ☐ Add **expected-vs-actual framing** (5–15% cold-reply prior) so a quiet month reads as “keep going.”
- ☐ Add **CSV export** of the core tables (your data, portable).

Definition of done. Opening the app immediately answers “what now?”, and the funnel shows honest conversion.

Phase 10 · Label Discovery

Goal	Browse the curated library to find your next target and pull it into your CRM.
Depends on	Phases 3 & 0 (populated labels).
Done when	You can browse/filter labels and add one to your CRM.

TASKS

- ☐ Build the **browse UI** over the read-only labels master.
- ☐ Add **filters**: genre, BPM range, submission method, tier, access_path, last_verified.
- ☐ Add a **“Have I submitted?”** indicator (join to your submissions).
- ☐ Add **freshness/stale warnings** when last_verified is past the threshold.
- ☐ Build **“Add to my CRM”** — copy the label into contacts with a label_id back-reference.
- ☐ Flag **“source updated — review”** on your CRM copy when the master changes.
- ☐ Show **sub-label reach hints** via parent_label_id (“this 1 send reaches 11”).

Definition of done. Filter the library to your next target and add it to your CRM in one click.

Phase 11 · Demo Link Tracking (optional)

Lower priority — unlisted SoundCloud already gives a free listen signal. Build only if you want automated open data.

Goal	See when a label opens a Form/DM demo link.
Depends on	Phase 4 (and the Edge runtime).
Done when	You can see when a label opens your Form/DM demo.

TASKS

- ☐ Create the link_events table.
- ☐ Build the **Edge Function redirect** at track.demotrack.app/{hash}.
- ☐ Generate a **unique hash per submission — Form/DM links only, never email**.
- ☐ Log each click: timestamp, repeat-open count, approximate country (IP geo).
- ☐ Feed **“seen, no reply”** triggers into the follow-up queue.
- ☐ Show open/repeat counts on the submission detail.

Definition of done. Opening a tracked Form/DM link is logged and can trigger a smart follow-up.

Phase 12 · Work Sessions + Goals

Goal	Gentle accountability — with progress derived, not typed.
Depends on	Phase 4 (submissions to count).
Done when	You can log studio time and track progress against targets.

WORK SESSIONS

- ☐ Build a **quick-add session**: hours + a **one-tap mood** (fire / good / ok / low).
- ☐ **Auto-derive “demos sent that day”** from submissions — don't ask the user.
- ☐ Compute and show a **session streak**.

GOALS

- ☐ Build goals: type (send / produce / response / growth / release / network), timeframe (monthly / quarterly / yearly), target.
- ☐ **Derive progress** with a Postgres view over submissions / tracks — the user only sets the target.
- ☐ Show progress bars on the dashboard.

Definition of done. Log a session in seconds and watch goal progress fill itself from real activity.

Phase 13 · Artist Press Kit

Goal	A shareable public EPK that attaches itself to every demo.
Depends on	Phases 1 & 5 (AI) & Storage.
Done when	You have a shareable EPK that auto-attaches to demos.

TASKS

- ☐ Build **press_kit CRUD**: artist name, location, genres, bio, past releases, links (SoundCloud/Beatport/Instagram), follower/play counts, photo, slug, is_public.
- ☐ Add **photo upload** to Supabase Storage (the only hosted asset).
- ☐ Wire **AI bio generation** in 3 tones (facts-only, no invented stats).
- ☐ Keep stats **manual** with an **“updated {date}”** stamp.
- ☐ Build the **public page** at `press.demotrack.app/{slug}`, served by the **whitelisted slug RPC** (no direct table reads).
- ☐ Wire the **auto-attach toggle** into Send Demo (Phase 4).

Definition of done. A clean public EPK exists at your slug and attaches its link to every demo by default.

Appendix · Build sequencing notes

WHY THIS ORDER

- **Usable early:** Phases 1–4 get you to a working send-and-record tool; everything after makes each send smarter.
- **Each phase ships:** deploy at the end of every phase — no big-bang release.
- **Parallelism:** Phase 0 (research) runs the entire time; the app is useless without the database, and the database takes real time to build.

HARD DEPENDENCIES AT A GLANCE

Phase	Cannot start until
4 Send Demo	2 Track Vault + 3 Contacts CRM
5 AI Hook	4 Send Demo
6 Follow-up	4 Send Demo (submissions exist)
7 Feedback	6 Follow-up (one-tap logging)
8 Intel + Prioritization	3 Contacts + 5 (to feed the hook)
9 Dashboard	4 + 6 + 7 (signals to aggregate)
10 Discovery	0 (populated labels) + 3
13 Press Kit	1 + 5 (AI bio) + Storage

CARRY THESE PRINCIPLES INTO EVERY PHASE

- **Derive, don't ask** — compute counts and progress; never make the user type a number.
- **One-tap over typing** — free text only for the hook, A&R notes, and feedback.
- **Silence is data** — overdue sends log themselves.
- **Structural privacy** — public data only ever leaves through the whitelisted slug endpoint.